

FDS lab exercise 1

B.Shashwath
CSE-K
2116250701697

Exp No:1.a Analyze the trend of data science job postings over the last decade

Description: Use web scraping (e.g., BeautifulSoup) or APIs (e.g., LinkedIn API) to gather data on the number of data science job postings each year. Use pandas for data manipulation and matplotlib/seaborn for visualization.

```
import pandas as pd #exp.no.1a
import matplotlib.pyplot as plt
import seaborn as sns
data = {'year': [2014, 2015, 2016, 2017, 2018, 2019, 2020, 2021, 2022, 2023], 'job_postings_count': [5000, 7500, 12000, 18000, 25000, 35000, 32000, 45000, 60000, 70000]}
df_jobs = pd.DataFrame(data)
print("Using dummy data for demonstration purposes.")
display(df_jobs.head())

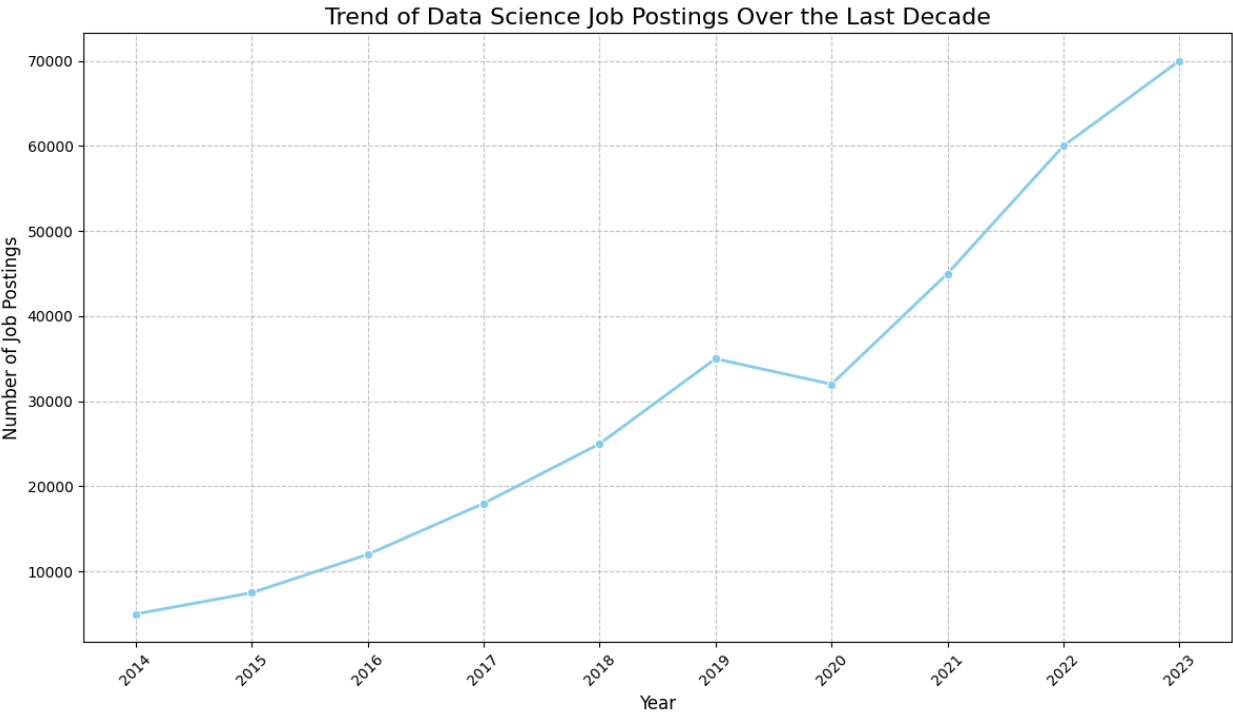
df_jobs['year'] = pd.to_numeric(df_jobs['year'], errors='coerce')
df_jobs.dropna(subset=['year'], inplace=True)
df_jobs['year'] = df_jobs['year'].astype(int)
if 'job_postings_count' in df_jobs.columns:
    job_trends = df_jobs.groupby('year')['job_postings_count'].sum().reset_index()
else:
    print("Warning: 'job_postings_count' column not found. Assuming each row is a job posting.")
    job_trends = df_jobs.groupby('year').size().reset_index(name='job_postings_count')

print("Aggregated job postings by year:")
display(job_trends.head())
plt.figure(figsize=(12, 7))
sns.lineplot(x='year', y='job_postings_count', data=job_trends, marker='o', color='skyblue', linewidth=2)
plt.title('Trend of Data Science Job Postings Over the Last Decade', fontsize=16)
plt.xlabel('Year', fontsize=12)
plt.ylabel('Number of Job Postings', fontsize=12)
plt.grid(True, linestyle='--', alpha=0.7)
plt.xticks(job_trends['year'], rotation=45)
plt.tight_layout()
plt.show()
```

	year	job_postings_count
0	2014	5000
1	2015	7500
2	2016	12000
3	2017	18000
4	2018	25000

Aggregated job postings by year

	year	job_postings_count
0	2014	5000
1	2015	7500
2	2016	12000
3	2017	18000
4	2018	25000



Exp No:1.b Analyze and visualize the distribution of various data science roles (Data Analyst, Data Engineer, Data Scientist, etc.) from a dataset.

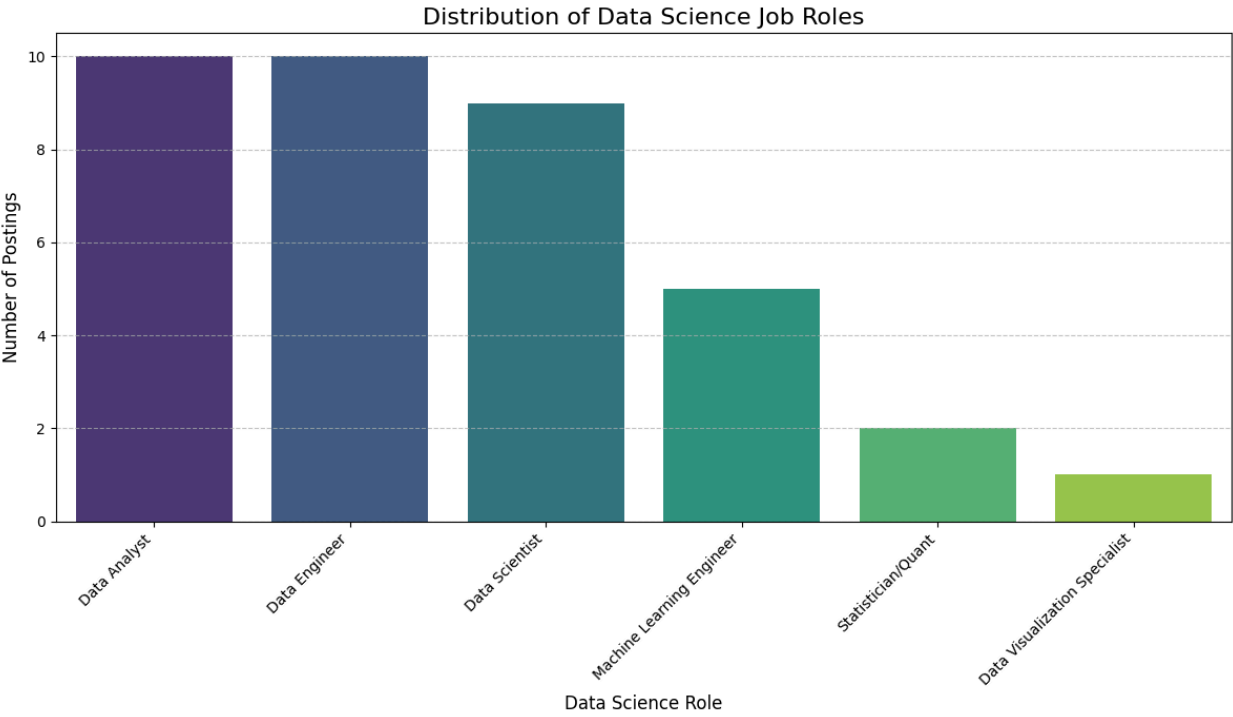
Description: Use a dataset of job postings and categorize them into different roles. Visualize the distribution using pie charts or bar plots.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
dummy_job_data = []
    {
        'job_title': [
            'Data Scientist',
            'Senior Data Scientist',
            'Junior Data Scientist',
            'Data Analyst',
            'Senior Data Analyst',
            'Business Intelligence Analyst',
            'Data Engineer',
            'Lead Data Engineer',
            'Machine Learning Engineer',
            'ML Engineer',
            'Research Scientist (AI)',
            'AI Engineer',
            'Statistician',
            'Quant Analyst',
            'Data Architect',
            'Analytics Engineer',
            'Big Data Engineer',
            'Data Visualization Specialist',
            'Data Scientist',
            'Data Analyst',
            'Data Engineer',
            'Machine Learning Engineer',
            'Data Scientist',
            'Data Analyst',
            'Data Engineer',
            'Data Scientist',
            'Data Analyst',
            'Data Engineer',
            'Data Scientist',
            'Data Analyst',
            'Data Engineer',
            'Data Scientist',
            'Data Analyst',
            'Data Engineer',
            'Data Scientist',
            'Data Analyst',
            'Data Engineer'
        ]
    }
df_roles = pd.DataFrame(dummy_job_data)
def categorize_role(job_title):
    job_title_lower = job_title.lower()
    if 'data scientist' in job_title_lower:
        return 'Data Scientist'
    elif 'data engineer' in job_title_lower or 'big data engineer' in job_title_lower or 'data architect' in job_title_lower:
        return 'Data Engineer'
    elif 'data analyst' in job_title_lower or 'business intelligence' in job_title_lower or 'analytics engineer' in job_title_lower:
        return 'Data Analyst'
    elif 'machine learning engineer' in job_title_lower or 'ml engineer' in job_title_lower or 'ai engineer' in job_title_lower or 'research scientist (ai)' in job_title_lower:
        return 'Machine Learning Engineer'
    elif 'statistician' in job_title_lower or 'quant analyst' in job_title_lower:
        return 'Statistician/Quant'
    elif 'data visualization' in job_title_lower:
        return 'Data Visualization Specialist'
    else:
        return 'Other'
df_roles['categorized_role'] = df_roles['job_title'].apply(categorize_role)
role_distribution = df_roles['categorized_role'].value_counts().reset_index()
role_distribution.columns = ['Role', 'Count']

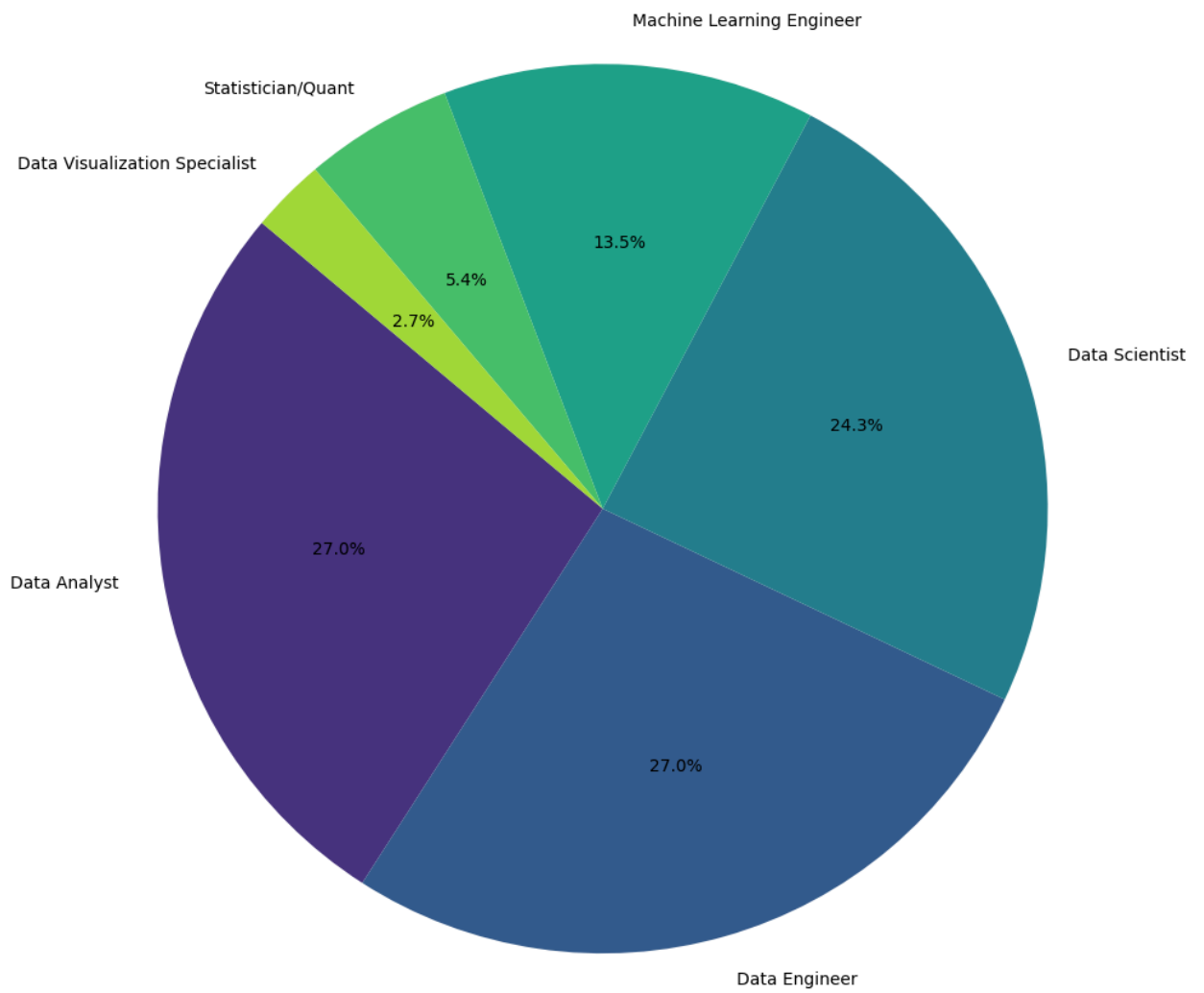
print("Distribution of Data Science Roles:")
display(role_distribution.head())
plt.figure(figsize=(12, 7))
sns.barplot(x='Role', y='Count', data=role_distribution, palette='viridis')
plt.title('Distribution of Data Science Job Roles', fontsize=15)
plt.xlabel('Data Science Role', fontsize=12)
plt.ylabel('Number of Postings', fontsize=12)
plt.xticks(rotation=45, ha='right')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
plt.figure(figsize=(10, 10))
plt.pie(role_distribution['Count'], labels=role_distribution['Role'], autopct='%1.1f%%', startangle=140, colors=sns.color_palette('viridis', len(role_distribution)))
plt.title('Percentage Distribution of Data Science Job Roles', fontsize=15)
plt.axis('equal') # Equal aspect ratio ensures that pie is drawn as a circle.
plt.tight_layout()
plt.show()
```

Distribution of Data Science Roles:

	Role	Count
0	Data Analyst	10
1	Data Engineer	10
2	Data Scientist	9
3	Machine Learning Engineer	5
4	Statistician/Quant	2



Percentage Distribution of Data Science Job Roles



Exp No:1.c Conduct an experiment to differentiate Structured , Un-structured and Semi structured data based on data sets given.

Description: Create small datasets for each type and explain their characteristics.

```
import pandas as pd #exp.no.1c
import json
data_structured = {'CustomerID': [1, 2, 3, 4], 'Name': ['Alice', 'Bob', 'Charlie', 'Diana'], 'Age': [30, 24, 35, 29], 'City': ['New York', 'Los Angeles', 'Chicago', 'Houston']}
df_structured = pd.DataFrame(data_structured)
print("Structured Data (Pandas DataFrame):")
display(df_structured)
data_semi_structured = [{"product_id": "P001", "name": "Laptop Pro", "price": 1200.00, "features": ["16GB RAM", "512GB SSD"], "reviews": [{"user": "Alice", "rating": 5, "comment": "Great performance!"}]},
                        {"product_id": "P002", "name": "Wireless Mouse", "price": 25.50, "color": "Black" }]
print("Semi-Structured Data (JSON):")
print(json.dumps(data_semi_structured, indent=2))
data_unstructured = ["The quick brown fox jumps over the lazy dog. This is a classic pangram.", "Customer feedback: The product arrived late and was slightly damaged. Overall, not a great experience.",
                    "A picture is worth a thousand words. Image analysis is a complex task.",
                    "Here's a tweet about data science: #datascience #AI #MachineLearning exciting field!"]
print("Unstructured Data (List of Text Strings):")
for i, doc in enumerate(data_unstructured):
    print(f"Document {i+1}: {doc}")
```

Structured Data (Pandas DataFrame):

	CustomerID	Name	Age	City
0	1	Alice	30	New York
1	2	Bob	24	Los Angeles
2	3	Charlie	35	Chicago
3	4	Diana	29	Houston

Semi-Structured Data (JSON):

```
[
  {
    "product_id": "P001",
    "name": "Laptop Pro",
    "price": 1200.0,
    "features": [
      "16GB RAM",
      "512GB SSD"
    ],
    "reviews": [
      {
        "user": "Alice",
        "rating": 5,
        "comment": "Great performance!"
      }
    ]
  },
  {
    "product_id": "P002",
    "name": "Wireless Mouse",
    "price": 25.5,
    "color": "Black"
  }
]
```

Unstructured Data (List of Text Strings):

Document 1: The quick brown fox jumps over the lazy dog. This is a classic pangram.

Document 2: Customer feedback: The product arrived late and was slightly damaged. Overall, not a great experience.

Document 3: A picture is worth a thousand words. Image analysis is a complex task.

Document 4: Here's a tweet about data science: #datascience #AI #MachineLearning exciting field!

Exp No:1.d Conduct an experiment to encrypt and decrypt given sensitive data.

Description: Use the cryptography library to encrypt and decrypt a piece of data.

Plain Text → Encryption → Cipher Text → Decryption → Plain Text

```
from cryptography.fernet import Fernet #exp.no.id
key = Fernet.generate_key()
fernet = Fernet(key)
print("Generated Encryption Key:", key.decode())
print("Note: In a real application, this key should be stored securely, not printed or hardcoded.")
sensitive_data = "My secret message: This information is highly confidential."
plain_text_bytes = sensitive_data.encode('utf-8')
cipher_text = fernet.encrypt(plain_text_bytes)
print("Original Plain Text:", sensitive_data)
print("Encrypted Cipher Text:", cipher_text.decode())
decrypted_text_bytes = fernet.decrypt(cipher_text)
decrypted_text = decrypted_text_bytes.decode('utf-8')
print("Encrypted Cipher Text (for reference):")
print(cipher_text.decode())
print("\nDecrypted Plain Text:")
print(decrypted_text)
if sensitive_data == decrypted_text:
    print("\nVerification: Decryption successful! The original and decrypted texts match.")
else:
    print("\nVerification: Decryption failed. The original and decrypted texts do NOT match.")
```

Generated Encryption Key: LOdUdGnMwheEuUvf7BV12HdPxri7lW2hnTQYG8Kss=

Note: In a real application, this key should be stored securely, not printed or hardcoded.

Original Plain Text: My secret message: This information is highly confidential.

Encrypted Cipher Text: gAAAAABqe-zyErfltdxMDJtk_GjnD11cLriZy69ZE9tBW-orjFw0Wozikoebg0JNrU5Tra9H4V2qdhfshw6qwmn4Bc6VdEWUdMbdySdbXUgwhU6pN3CwhCFGUXHfUeuYbJVyb86I1xKqc14mApgyBKmmI1TE6JF7_w==

Encrypted Cipher Text (for reference):

gAAAAABqe-zyErfltdxMDJtk_GjnD11cLriZy69ZE9tBW-orjFw0Wozikoebg0JNrU5Tra9H4V2qdhfshw6qwmn4Bc6VdEWUdMbdySdbXUgwhU6pN3CwhCFGUXHfUeuYbJVyb86I1xKqc14mApgyBKmmI1TE6JF7_w==

Decrypted Plain Text:

My secret message: This information is highly confidential.

Verification: Decryption successful! The original and decrypted texts match.